

12 – CRUD Operations & Data Modeling in NoSQL

DATABASE SYSTEM

Vincentius Kurniawan S.Kom., M.Eng.Sc.,

COURSE OBJECTIVES

GOALS.

IF-351 – 2025

- 1 [Adding](#) new documents to a collection
- 2 [Removing](#) documents from a collection
- 3 [Updating](#) existing documents
- 4 To [Embed](#) or [Reference](#) documents



OUTLINES.

- Inserting Documents: `insertOne`, `insertMany`
- Insert Validation
- Removing Document: `drop`
- Updating Document: `updateOne`
- Upsert
- Updating Multiple Document: `updateMany`
- Embedding for Locality, Atomicity, and Isolation
- Referencing for Flexibility

INSERTING DOCUMENT

Inserts are the basic method for adding data to MongoDB. To insert a single document, use the collection's `insertOne` method:

```
> db.movies.insertOne({"title" : "Stand by Me"})
```

`insertOne` will add an `"_id"` key to the document (if you do not supply one) and store the document in MongoDB

```
IF351> db.movies.insertOne({"title":"Stand by Me"})
{
  acknowledged: true,
  insertedId: ObjectId('685904fbc8220097b96c4bd0')
}
```

```
IF351> db.movies.find()
[
  {
    _id: ObjectId('685904fbc8220097b96c4bd0'),
    title: 'Stand by Me' },
  {
    _id: ObjectId('6859069ac8220097b96c4bd1'),
    title: 'Mission Impossible'
  }
]
```

INSERTING MULTIPLE DOCUMENT

If you need to insert **multiple documents** into a collection, you can use `insertMany`. This method enables you to pass an array of documents to the database. This is far more efficient because your code will not make a round trip to the database for each document inserted but will insert them in bulk.

```
IF351> db.movies.insertMany([
  {"title" : "Ghostbusters"},
  {"title" : "E.T."},
  {"title" : "Blade Runner"}]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId( '685909acc8220097b96c4bd2' ),
    '1': ObjectId( '685909acc8220097b96c4bd3' ),
    '2': ObjectId( '685909acc8220097b96c4bd4' )
  }
}
```

INSERTING MULTIPLE DOCUMENT (CONT.D)

In this example, because [ordered inserts is the default](#), only the first two documents will be inserted. The third document will produce an error, because you cannot insert two documents with the same "_id":

```
IF351> db.movies.insertMany([
...   { "_id" : 0, "title" : "Top Gun"},
...   { "_id" : 1, "title" : "Back to the Future"},
...   { "_id" : 1, "title" : "Gremlins"},
...   { "_id" : 2, "title" : "Aliens"}])
Uncaught:
MongoBulkWriteError: E11000 duplicate key error collection: IF351.movies index: _id_ dup key: { _id: 1 }
Result: BulkWriteResult {
  insertedCount: 2,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: { '0': 0, '1': 1 }
}
Write Errors: [
  WriteError {
    err: {
      index: 2,
      code: 11000,
      errmsg: 'E11000 duplicate key error collection: IF351.movies index: _id_ dup key: { _id: 1 }',
      errInfo: undefined,
      op: { _id: 1, title: 'Gremlins' }
    }
  }
]
IF351> db.movies.find()
[
  { _id: 0, title: 'Top Gun' },
  { _id: 1, title: 'Back to the Future' }
]
```

INSERTING MULTIPLE DOCUMENT (CONT.D)

If instead we specify [unordered inserts](#), the first, second, and fourth documents in the array are inserted. The only insert that fails is the third document, again because of a duplicate "_id" error:

```
IF351> db.movies.insertMany([
...   { "_id" : 3, "title" : "Sixteen Candles"},
...   { "_id" : 4, "title" : "The Terminator"},
...   { "_id" : 4, "title" : "The Princess Bride"},
...   { "_id" : 5, "title" : "Scarface"}],
...   {"ordered" : false})
Uncaught:
MongoBulkWriteError: E11000 duplicate key error collection: IF351.movies index: _id_dup key: { _id: 4 }
Result: BulkWriteResult {
  insertedCount: 3,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: { '0': 3, '1': 4, '3': 5 }
}
Write Errors: [
  WriteError {
    err: {
      index: 2,
      code: 11000,
      errmsg: 'E11000 duplicate key error collection: IF351.movies index: _id_dup key: { _id: 4 }',
      errInfo: undefined,
      op: { _id: 4, title: 'The Princess Bride' }
    }
  }
]
IF351> db.movies.find()
[
  { _id: 3, title: 'Sixteen Candles' },
  { _id: 4, title: 'The Terminator' },
  { _id: 5, title: 'Scarface' }
]
```

INSERT VALIDATION

MongoDB does minimal checks on data being inserted such as:

- the document's basic structure
- adds an "_id" field if one does not exist
- all documents must be [smaller than 16 MB](#)
- documents that are too large
- Validation of non-UTF-8 strings
- Is there any unrecognized types

To see the Binary JSON (BSON) size, in bytes, of the document doc, run `bsonsize(doc)` from the shell.

```
IF351> bsonsize(db.movies.findOne({_id:3}))  
41
```



```

IF351> db.movies.find()
[
  { _id: 3, title: 'Sixteen Candles' },
  { _id: 4, title: 'The Terminator' },
  { _id: 5, title: 'Scarface' },
  { _id: 1, title: 'Harry Potter' },
  { _id: 2, title: 'Mission Impossible' }
]
IF351> db.movies.deleteOne({_id:4})
{ acknowledged: true, deletedCount: 1 }
IF351> db.movies.find()
[
  { _id: 3, title: 'Sixteen Candles' },
  { _id: 5, title: 'Scarface' },
  { _id: 1, title: 'Harry Potter' },
  { _id: 2, title: 'Mission Impossible' }
]
IF351>

```

```

IF351> movies.find()
{ "_id" : 0, "title" : "Top Gun", "year" : 1986 }
{ "_id" : 1, "title" : "Back to the Future", "year" : 1985 }
{ "_id" : 3, "title" : "Sixteen Candles", "year" : 1984 }
{ "_id" : 4, "title" : "The Terminator", "year" : 1984 }
{ "_id" : 5, "title" : "Scarface", "year" : 1983 }

IF351> db.movies.deleteMany({"year" : 1984})
{ "acknowledged" : true, "deletedCount" : 2 }

IF351> db.movies.find()
{ "_id" : 0, "title" : "Top Gun", "year" : 1986 }
{ "_id" : 1, "title" : "Back to the Future", "year" : 1985 }
{ "_id" : 5, "title" : "Scarface", "year" : 1983 }

```

DELETE ONE

REMOVING DOCUMENT(S)

`deleteOne` method take a filter document as their first parameter. The filter specifies a set of criteria to match against in removing documents. To delete the document with the `"_id"` value of 4, we use `deleteOne` in the mongo shell as illustrated

To delete all the documents that match a filter, use `deleteMany`.

```
IF351> db.movies.find()  
{ "_id" : 0, "title" : "Top Gun", "year" : 1986 }  
{ "_id" : 1, "title" : "Back to the Future", "year" : 1985 }  
{ "_id" : 3, "title" : "Sixteen Candles", "year" : 1984 }  
{ "_id" : 4, "title" : "The Terminator", "year" : 1984 }  
{ "_id" : 5, "title" : "Scarface", "year" : 1983 }  
  
IF351> db.movies.deleteMany({})  
{ "acknowledged" : true, "deletedCount" : 5 }  
  
IF351> db.movies.find()
```

```
IF351> db.movies.drop()  
true
```

DROP

REMOVING COLLECTION

It is possible to use `deleteMany` to remove all documents in a collection. Removing documents is usually a fairly quick operation. However, if you want to clear an entire collection, it is faster to `drop` it, and then recreate any indexes on the empty collection. Once data has been removed, it is gone forever. There is no way to undo a delete or drop operation or recover deleted documents.

DOCUMENT REPLACEMENT

UPDATING DOCUMENT

Once a document is stored in the database, it can be changed using one of several update methods: `updateOne`, `updateMany`, and `replaceOne`. `updateOne` and `updateMany` each take a [filter document](#) as their first parameter and a [modifier document](#), which describes changes to make, as the second parameter

```
1 //Previous Structure
2 {
3   "_id" : ObjectId("4b2b9f67a1f631733d917a7a"),
4   "name" : "joe",
5   "friends" : 32,
6   "enemies" : 2
7 }
8
9 //Steps
10 > var joe = db.users.findOne({"name" : "joe"});
11
12 > joe.relationships = {"friends" : joe.friends, "enemies" : joe.enemies};
13 {
14   "friends" : 32,
15   "enemies" : 2
16 }
17
18 > joe.username = joe.name;
19 "joe"
20
21 > delete joe.friends;
22 true
23
24 > delete joe.enemies;
25 true
26
27 > delete joe.name;
28 true
29
30 > db.users.replaceOne({"name" : "joe"}, joe);
31
32 //After Replaced
33 {
34   "_id" : ObjectId("4b2b9f67a1f631733d917a7a"),
35   "username" : "joe",
36   "relationships" : {
37     "friends" : 32,
38     "enemies" : 2
39 }
```

UPDATING DOCUMENT (CONT.D)



A common mistake is matching more than one document with the criteria and then creating a duplicate "_id" value with the second parameter. The database will throw an error for this, and no documents will be updated.

For example, suppose we create several documents with the same value for "name":

```
1 > db.people.find()
2 { "_id" : ObjectId("4b2b9f67a1f631733d917a7b"), "name" : "joe", "age" : 65}
3 { "_id" : ObjectId("4b2b9f67a1f631733d917a7c"), "name" : "joe", "age" : 20}
4 { "_id" : ObjectId("4b2b9f67a1f631733d917a7d"), "name" : "joe", "age" : 49}
```

Now, if it's Joe #2's birthday, we want to increment the value of his "age" key, so we might say this:

```
1 > joe = db.people.findOne({"name" : "joe", "age" : 20});
2 {
3   "_id" : ObjectId("4b2b9f67a1f631733d917a7c"),
4   "name" : "joe",
5   "age" : 20
6 }
7 > joe.age++;
8 > db.people.replaceOne({"name" : "joe"}, joe);
9 E11001 duplicate key on update
```

USING UPDATE OPERATORS

Usually only certain portions of a document need to be updated. You can update specific fields in a document using atomic *update operators*. Update operators are special keys that can be used to specify complex update operations, such as [altering](#), [adding](#), or [removing keys](#), and [even manipulating arrays](#) and [embedded documents](#)

For example, we're keeping website analytics in a collection and want to increment a counter each time someone visits a page

```
1 {  
2   "_id" : ObjectId("4b253b067525f35f94b60a31"),  
3   "url" : "www.example.com",  
4   "pageviews" : 52  
5 }
```

Every time someone visits a page, we can find the page by its URL and use the "\$inc" modifier to increment the value of the "pageviews" key:

```
1 > db.analytics.updateOne(  
2   "url" : "www.example.com"  
3 },{  
4   "$inc" : {"pageviews" : 1}  
5 })  
6 { "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }  
7  
8 //Result  
9 > db.analytics.findOne()  
10 {  
11   "_id" : ObjectId("4b253b067525f35f94b60a31"),  
12   "url" : "www.example.com",  
13   "pageviews" : 53  
14 }
```

USING UPDATE OPERATORS (CONT.D)

"\$set" sets the value of a field. If the field does not yet exist, it will be created. This can be handy for updating schemas or adding user-defined keys. "\$set" can even change the type of the key it modifies (i.e From string to Array)

You can remove the key altogether with \$unset

```
1 > db.users.updateOne({"name" : "joe"},
2 ... {"$set" : {"favorite book" : "Green Eggs and Ham"}})
3
4 > db.users.updateOne({"name" : "joe"},
5 ... {"$unset" : {"favorite book" : 1}})
```

USING UPDATE OPERATORS (CONT.D)

"\$set" sets the value of a field. If the field does not yet exist, it will be created. This can be handy for updating schemas or adding user-defined keys. "\$set" can even change the type of the key it modifies (i.e From string to Array)

You can remove the key altogether with \$unset

```
1 > db.users.updateOne({"name" : "joe"},
2 ... {"$set" : {"favorite book" : "Green Eggs and Ham"}})
3
4 > db.users.updateOne({"name" : "joe"},
5 ... {"$unset" : {"favorite book" : 1}})
```

USING UPDATE OPERATORS (CONT.D)

An extensive class of update operators exists for manipulating arrays.

- `$push` → adds elements to the end of an array if the array exists and creates a new array if it does not
- `$each` → Used with `$push` to insert multiple items at once into an array.
- `$slice` → Limits the number of elements in an array after a `$push`. Keeps newest or oldest.
- `$sort` → Sorts array elements after pushing, based on a field or value.
- `$addToSet` → Adds a value to the array only if it doesn't already exist (deduplication)
- `$pull` → Removes matching elements from an array (e.g. by value, condition).
- `$pop` Removes either the first (-1) or last (1) element from an array

```
1 db.products.updateOne(  
2   { _id: 1 },  
3   {  
4     $push: {  
5       reviews: {  
6         $each: [{ rating: 5 }, { rating: 3 }],  
7         $slice: -5,  
8         $sort: { rating: -1 }  
9       }  
10    }  
11  }  
12 )
```


POSITIONAL ARRAY MODIFICATION

The `$` symbol is used in update queries to indicate the first matching element in an array. It works with update operators like `$set`, `$inc`. It must match the array element in the query filter to use it.

Let's say you have a document like this:

```
1 {  
2   _id: 1,  
3   items: [  
4     { name: "apple", qty: 5 },  
5     { name: "banana", qty: 7 }  
6   ]  
7 }
```

To update the qty of the item named "banana":

```
1 db.inventory.updateOne(  
2   { _id: 1, "items.name": "banana" },  
3   { $set: { "items.$.qty": 10 } }  
4 )
```

ARRAY FILTER

The limitation of positional array modification is only worked for single match (the first matched element). To update multiple matching elements, we can use `arrayFilter`

Let's say you have a document like this:

```
1 {  
2   _id: 1,  
3   items: [  
4     { name: "pen", qty: 2 },  
5     { name: "pencil", qty: 7 },  
6     { name: "eraser", qty: 3 }  
7   ]  
8 }  
9
```

To update the qty of the item which are <5

```
1 db.orders.updateOne(  
2   { _id: 1 },  
3   { $set: { "items.$[elem].qty": 5 } },  
4   arrayFilters: [ { "elem.qty": { $lt: 5 } } ] }  
5 )  
6
```

Result:

```
1 {  
2   _id: 1,  
3   items: [  
4     { name: "pen", qty: 5 },  
5     { name: "pencil", qty: 7 },  
6     { name: "eraser", qty: 5 }  
7   ]  
8 }
```

UP SERT

UPDATE AND INSERT

An *upsert* is a special type of update. If no document is found that matches the filter, a new document will be created by combining the criteria and updated documents. If a matching document is found, it will be updated normally. *Upserts* can be handy because they can eliminate the need to “seed” your collection: you can often have the same code create and update documents.

```
1 db.users.updateOne(  
2   { username: "vincentius" },  
3   { $set: { lastLogin: new Date() } },  
4   { upsert: true }  
5 )
```

UPDATE MANY

`updateMany` provides a powerful tool for performing schema migrations or rolling out new features to certain users.

Suppose, for example, we want to give a gift to every user who has a birthday on a certain day. We can use `updateMany` to add a "gift" to their accounts. For example:

```
1 > db.users.insertMany([
2   ... {birthday: "10/13/1978"},
3   ... {birthday: "10/13/1978"},
4   ... {birthday: "10/13/1978"}])
5
6 {
7   "acknowledged" : true,
8   "insertedIds" : [
9     ObjectId("5727d6fc6855a935cb57a65b"),
10    ObjectId("5727d6fc6855a935cb57a65c"),
11    ObjectId("5727d6fc6855a935cb57a65d")
12  ]
13 }
14
15 > db.users.updateMany(
16   {"birthday" : "10/13/1978"},
17   {"$set" : {"gift" : "Happy Birthday!"}})
18
19 { "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 3 }
```

PROBLEM WITH NORMALIZATION

The thing about normalization is that it allows for [easy updating](#) without any [redundancy](#). Each fact about the application domain can be updated by changing just one value, at one row-column intersection. The problem arises when you try to get the data *back out* especially if it must use JOIN.

[joining tables is one of the most expensive operations](#) that relational databases do. Additionally, if you end up needing to scale your database to multiple servers, you introduce the problem of generating a distributed join, [a complex and generally slow operation](#).

```
SELECT name, phone_number
FROM contacts LEFT JOIN numbers
ON contacts.contact_id=numbers.contact_id
WHERE contacts.contact_id=3;
```

DENORMALIZING FOR PERFORMANCE

Denormalize the model is usually conducted to reduce the number of JOIN operations required for the queries we execute frequently.

The trade offs: denormalization will results in redundancy we were trying to get away from, and leads to greater application complexity, as we have to make sure to update data in all its redundant locations.

In MongoDB, both normalized and denormalized approaches can be used depending on the use case. Embedding is commonly used when related data is stored directly inside the parent document as a nested sub-document or array. this represents a denormalized design.

Meanwhile, MongoDB also supports referencing, where only the `_id` (or foreign key) of related data is stored in the document. The full details are retrieved through a separate query. This is a normalized approach that promotes separation of concerns and data reuse.

EMBEDDING FOR LOCALITY

Embedding is preferred in [one-to-many relationships](#) when related data is often accessed together because MongoDB stores documents contiguously on disk—leading to efficient sequential reads. Since [MongoDB doesn't support traditional JOINS](#), referencing introduces multiple disk seeks and additional round-trip latency.

For example, fetching a contact and its phone numbers via references requires two separate queries. This is slower than embedding, where all data resides in a single document. So, if your app frequently needs both parent and child data together, embedding improves locality and performance.

```
{
  _id: ObjectId("64fa12abcdef123456789012"),
  name: "Alice",
  email: "alice@example.com",
  phoneNumbers: [
    { type: "mobile", number: "+62-812-3456-7890" },
    { type: "home", number: "+62-21-555-1234" },
    { type: "work", number: "+62-21-789-4321" }
  ]
}
```



Works best when the number of sub-items (e.g. phone numbers) is bounded or relatively small.



Keep embedded arrays manageable in size to avoid document bloat and exceed size limits (16MB max).

EMBEDDING FOR ATOMICITY AND ISOLATION

Another reason to use embedding in MongoDB is to ensure [atomicity](#) and [isolation](#) during write operations. Unlike relational databases that support multi-statement transactions to guarantee consistency (e.g. DELETE operations across related tables), [MongoDB traditionally does not support multi-document transactions](#). This means that in a normalized schema using references, removing a parent and its children (e.g. a contact and their phone numbers) requires multiple operations—risking partial updates and visibility of incomplete data.

Embedding solves this by storing related data in one document, allowing a [single atomic operation](#) to manage the entire relationship safely.



```
// Reference-Based Delete (Risk of Partial Success)
// Separate deletes, not atomic
db.contacts.remove({ _id: 3 });
db.numbers.remove({ contact_id: 3 });
// Risk: If the second command fails or is delayed,
// numbers might remain orphaned.
```



```
// All related data in one document
{
  _id: 3,
  name: "Jenny",
  phoneNumbers: [
    { type: "home", number: "+62-21-1111" },
    { type: "mobile", number: "+62-812-2222" }
  ]
}

// Deletion is atomic
db.contacts.remove({ _id: 3 });
```


REFERENCING FOR FLEXIBILITY

While embedding offers performance and consistency, especially for one-way, predictable queries, it can become limiting if your application needs to [query data in flexible](#) or [multi-dimensional](#) ways.

In particular, embedding works well when you're displaying related data together (e.g., posts with comments), but becomes inefficient if you need to query deeply nested elements independently (e.g., search all comments by a specific user).

In such cases, a normalized schema with references across collections makes querying easier, more flexible, and allows for operations like [filtering](#), [sorting](#), and [pagination directly on subdocuments](#).



```
// Embedded Schema: Limited Query Flexibility
// posts collection
{
  _id: "First Post",
  author: "Rick",
  text: "This is my first post",
  comments: [
    { author: "Stuart", text: "Nice post!" },
    { author: "Mark", text: "Dislike!" }
  ]
}

//Query
db.posts.find({ "comments.author": "Stuart" }, { comments: 1 })

//Drawbacks: Returns all comments from matching posts—not just Stuart's.
```



```
// Normalized Schema: Better Query Control
// posts
{
  _id: "First Post",
  author: "Rick",
  text: "This is my first post"
}

// comments
{
  _id: ObjectId("abc123"),
  post_id: "First Post",
  author: "Stuart",
  text: "Nice post!"
}

//Query
db.comments.find({ author: "Stuart" })
// Returns only Stuart's comments, supports sorting, limiting, and pagination.
```

REFERENCING FOR POTENTIALLY HIGH-ARITY RELATIONSHIP

In cases where your one-to-many relationships involve large or [unpredictable numbers of related items \(high arity\)](#), like hundreds or thousands of comments per blog post, normalization becomes preferable over embedding due to these key concerns:

- RAM usage spikes: Large documents consume more memory, reducing overall caching efficiency.
- Update performance drops: MongoDB may need to relocate expanding documents to larger storage blocks, slowing down writes.
- Document size limit: MongoDB has a 16 MB max size per document, which can be breached with high-volume data.

Instead, [referencing](#) allows better scalability, query flexibility, and avoids bloated documents.



```
// Embedded (Not Ideal for Hundreds of Comments)
{
  _id: "First Post",
  title: "Scaling MongoDB",
  comments: [
    { author: "User1", text: "Great post!" },
    { author: "User2", text: "Thanks for sharing." },
    ...
  ]
  // 500+ comments cause RAM pressure and document growth
}
```



```
// Normalized (Scalable & Efficient)
// posts collection
{
  _id: "First Post",
  title: "Scaling MongoDB"
}

// comments collection
{
  _id: ObjectId("abc123"),
  post_id: "First Post",
  author: "User1",
  text: "Great post!"
}

// Lightweight post reads (e.g., display only top 3 comments)
// Paging, sorting, filtering directly on comments
// Avoiding RAM exhaustion or hitting document size limits
```

ADVANTAGE AND DISADVANTAGE

	Advantage	Disadvantage
EMBEDDING	- Faster read performance (no joins) - Great for data that's often queried together - Simpler structure and fewer queries	- Document size grows quickly - Harder to update nested structures if reused across documents - Limited flexibility for many-to-many relationship
REFERENCING	- Better separation of concerns and reuse - Ideal for large or shared data entities - Easier updates across multiple documents	- Requires additional queries (manual joins) - Slightly more complex querying logic - Slower read performance compared to embedding

EMBED VS REFERENCE

WHEN TO USE

Use Embedding	Use Referencing
One-to-few relationships	One-to-many or many-to-many
Data grows predictably & stays small	Reusable or frequently updated data
Query performance is critical	Data integrity and flexibility are key

SUMMARY

Adding New Documents

Use `insertOne()` or `insertMany()` to add documents to a MongoDB collection, each document is a flexible JSON-like structure

Removing Document

Delete documents using `deleteOne()` for a single match or `deleteMany()` to remove multiple based on a query filter

Updating Existing Documents

Modify documents using `updateOne()` or `updateMany()` combined with update operators like `$set`, `$unset`, or `$inc`.

Embed Document

Store related data as nested sub-documents or arrays within a parent document, ideal for tightly coupled, frequently accessed data.

Reference Documents

Link related documents using `_id` fields, storing references between collections—great for decoupled data and flexible queries.

REFERENCES

Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley

Van der Lans, R. F. (2007). SQL for MySQL developers. Addison Wesley.

Swan, M. (2015). Blockchain: Blueprint for a new economy. O'Reilly Media.

Bradshaw, D., Brazil, K., & Chodorow, K. (2019). MongoDB: The definitive guide—Powerful and scalable data storage. O'Reilly Media.

Copeland, R. (2013). MongoDB applied design patterns. O'Reilly Media.



NEXT OUTLINES

NEXT WEEK

- Querying
- Indexes
- Aggregation



KEEP IN TOUCH

THANK YOU

vincentius.kurniawan@lecturer.umn.ac.id



@aixeta#2683

